Machine Learning for Lithium-Ion Battery Remaining Useful Life Prediction Using Multivariate Degradation Data

Neha Varadharajan (240962214)
CSE AIML
School of Computer Engineering
Manipal Institute of Technology, Manipal

***Abstract*—Accurate estimation of the remaining useful life of lithium-ion batteries enables the maintenance and safe operation of battery powered systems. This work compares a cycle based reference, Random Forest, XGBoost and a unidirectional LSTM for discharge-cycle RUL regression on the NASA battery data set using four cells (B0005, B0006, B0007, B0018) in the audit and a nested leave-one-battery-out protocol. The end of life (EOL) is defined as the first measured discharge capacity at or below 1.4 Ah; B0007 never crosses this threshold and is treated as right-censored thereby leaving three observed-EOL cells for supervised scoring. Predictions are made using only current and past cycles in windows of 10, 20 and 30 cycles, with imputation and scaling fitted on the training data only, and full and reduced health-indicator sets are compared. The Random Forest model with all features and a 20-cycle history yielded the lowest macro RMSE of 8.986 cycles and MAE of 7.637 cycles, with XGBoost following closely at 9.002 cycles, compared to 14.063 cycles for the cycle baseline, a descriptive reduction of about 36.10%.A validation-driven tree-selection procedure achieves a macro RMSE of approximately 11.852 cycles. The removal of capacity and duration indicators uniformly worsens each matched model, and Tree SHAP explanations differ by held-out cell. With three scored cells and one seed, the results are descriptive and are not posited to be statistically significant or deployment-ready.**

***Index Terms*—Remaining useful life, lithium-ion battery, NASA battery data set, Random Forest, XGBoost, LSTM, SHAP, leave-one-battery-out validation, right-censoring.**

I. Introduction

Lithium-ion batteries are used in a large variety of applications, ranging from portable electronics to electric vehicles, and from traction energy storage systems to stationary energy storage systems. When subjected to repeated charging and discharging cycles, a battery cell is permanently discharged and no longer able to meet the application needs. The ability to estimate the number of operating cycles that the battery will endure before its RUL occurs (i.e., the remaining useful life) enables to schedule maintenance and/or battery replacement in advance and therefore provides a more valuable information than a mere present state-of-health indication.

Degradation measures are available from periodic characterization of each cell. In the data set used here, each discharge operation provides summaries of voltage, current, temperature and duration, together with the measured discharge capacity, while preceding charge operations and electrochemical impedance spectroscopy (EIS) measurements provide additional context [1]. Capacity is the most direct indicator of health, but it is not the only informative signal, and one practical question is how much predictive information remains when direct capacity-related inputs are withheld.

Data-driven remaining useful life prediction presents a number of methodological challenges. First, as cells age differently, the model must produce a generalization across batteries rather than from the rows of a single battery. Second, the consecutive cycles are highly correlated, which causes a significant overestimation of performance when randomly partitioning the windows [2]. Third, the most common definition of an EOL is a threshold crossing event, which means that a subset of cells does not actually fail within the period of observation, making them censored. Fourth, the validation procedure that uses the same cells for both model selection and error reporting produces an overly optimistic error estimate due to the hyperparameter misselection bias [3].

The purpose of this study was to design a controlled experiment that would address these challenges in order to obtain a realistic assessment of the RUL prediction algorithms. In this experiment, EOL is determined based on the raw measurements, with censored cells being excluded from analysis with no artificial labels applied. Additionally, the models are trained with no access to the labels of the current cycle, and each cell contributes to the testing set only once, with no overlap between training, validation, and test sets. The evaluation compares four different algorithms, including a naive approach of cycling averaging, Random Forest [4], XGBoost [5] and LSTM networks [6]. The experiment is conducted with different history lengths (10, 20, and 30 cycles) and sets of features, with the Tree SHAP values [7], [8] being calculated for interpretability. The work is limited to a small-scale proof-of-concept study with only a few cells, and the conclusions are drawn within the context of this experiment.

II. Literature Survey

The literature on battery lifetime prediction spans physics-based modelling, statistical regression, classical machine learning and deep sequence models. The following subsections outline the lines of work most relevant to this study.

A. Battery Degradation and RUL Prediction

A cell's degradation path can be considered to carry information about the number of cycles remaining until a certain failure criterion is reached. The definition of EOL based on a capacity threshold therefore requires special consideration for cells that do not reach the prescribed threshold during the observation time. For those, the lifespan is only known to be higher than the length of observation.

B. Data-Driven Prediction from Cycling Data

Severson et al. [9] have shown that features derived from discharge voltage curves in early cycles can be used in conjunction with regularized linear models to predict battery cycle life before capacity degradation becomes apparent, thus demonstrating that besides capacity, there are features that carry predictive information. Richardson et al. [10] have applied Gaussian process regression to predict the state of health of batteries. These works motivate our analysis of whether indirect signals such as voltage, current, temperature and impedance retain predictive value when direct capacity inputs are removed, an issue we address with the reduced feature set.

C. Tree Ensembles for Tabular Features

Breiman [2] proposed random forests, which average many decision trees built on bootstrap samples in order to decrease variance. Chen & Guestrin [4] describe XGBoost, a scalable implementation of gradient boosted trees. Both methods can represent non-linear relationships between any set of engineered summary features, without requiring an explicit parametric form, and are thus natural choices for tabular health-indicator summaries.

D. Recurrent Models for Sequence Data

Long short-term memory networks [6] process sequences through gated recurrent units and are a common choice for degradation sequences. Zhang et al. [11] applied LSTM recurrent networks to the RUL prediction of lithium-ion batteries. In this study, a compact unidirectional LSTM is included as a sequence-model comparison against tree models that consume summaries of the same windows.

E. Censoring and End-of-Life Definition

When a cell has not crossed the EOL threshold by the end of its record, its true EOL is unknown; assigning its last observed cycle as EOL or an arbitrary extrapolation from that point would be assigning it a fabricated label. The cell is kept in the audit and visualization, but excluded from supervised training and scoring. Survival-analysis methods are the natural approach to learning from censored trajectories, but are not implemented here and are noted as future work.

F. Explainable AI and SHAP

Lundberg and Lee [7] introduced SHapley Additive exPlanations (SHAP), which assign each feature a contribution to an individual prediction using cooperative game theory, and Lundberg et al. [8] developed an efficient tree-specific algorithm. SHAP describes how a fitted model uses its inputs. It describes associations learned by the model rather than physical causation, and correlated features can share attribution.

G. Time-Series Validation and Leakage Prevention

Bergmeir and Benítez [2] examined cross-validation for time-series predictor evaluation and emphasized that evaluation must respect temporal dependence. For multi-battery data, the analogous requirement is to hold out whole batteries, so that no cycle of a test cell influences training. Cawley and Talbot [3] showed that selecting hyperparameters and reporting performance on the same data leads to selection bias, which motivates the nested design used here.

H. Positioning of the Present Study

The present study is a controlled small-scale comparative benchmark using the NASA battery data set [1]. It does not claim to outperform the more elaborate methods of the literature and achieve population-level accuracy. It provides an explicit censoring audit, past-only windows with common evaluation endpoints, nested battery-level validation, grouped feature ablation and fold-specific explanations, so that the comparison between models reflects the modelling choices rather than data handling artifacts.

III. Objectives

The problem is formulated as a regression task: given the history of a cell up to and including the current discharge cycle t, predict its RUL, defined as the number of discharge cycles remaining until the first measured capacity at or below 1.4 Ah. Errors are reported in discharge cycles. They are not percentage accuracies.

The objectives of this study are:

1) To build a leakage-aware pipeline that extracts discharge-level features from the NASA MATLAB files, joins them chronologically to preceding charge and impedance operations, defines EOL as the first observed threshold crossing, and audits censored cells.
2) To compare a cycle-based reference with Random Forest, XGBoost and an LSTM under nested leave-one-battery-out evaluation.
3) To study the effect of the history length (10, 20 and 30 cycles) with identical scored endpoints for all windows.
4) To quantify, through a grouped ablation, how much the models depend on direct capacity and duration indicators by comparing full and reduced feature sets.
5) To examine cross-cell behavior through per-cell errors, bias and prediction-versus-observation curves.
6) To use Tree SHAP to describe how the inner-selected tree models use their inputs on each held-out cell.

TABLE I
DATA AUDIT OF THE FOUR NASA CELLS (THRESHOLD 1.4 AH)

Cell	Cycles obs.	Last cap. (Ah)	Min cap. (Ah)	EOL cycle	Cens.	Excl. post-EOL
B0005	168	1.325079	1.287453	125	No	43
B0006	168	1.185675	1.153818	109	No	59
B0007	168	1.432455	1.400455	—	Yes	0
B0018	132	1.341051	1.341051	97	No	35

7) To demonstrate inference from a saved model and a partial history without supplying the future EOL to the predictor.

IV. DATA SOURCE

A. NASA Battery Data Set

The experiments are executed using the Battery Data Set of the NASA Prognostics Data Repository [1]. The project downloads the NASA-related archive, extracts the original MATLAB-structured files and parses four cells: B0005, B0006, B0007 and B0018. All four requested files are found available. Each discharge procedure is reduced to a record with the cycle index, discharge capacity, summaries of voltage (mean, standard deviation, minimum), mean current, mean and maximum temperature and discharge duration. A discharge is matched to the most recent preceding charge procedure (charge duration and mean charge voltage) and impedance measurement (ohmic resistance R_e, charge-transfer resistance R_{ct} and the age of the impedance measurement in cycles). Operation order in the source structure is assumed to be chronological, and no backward filling from future tests is used.

B. Data Audit

Table I summarizes the audit. In total, 636 discharge records were observed (168 + 168 + 168 + 132). The first observed threshold crossings are at cycle 125 for B0005, 109 for B0006 and 97 for B0018. The minimum observed capacity of B0007 is 1.400455 Ah, narrowly above the 1.4 Ah threshold, and it is therefore right-censored and is not rounded or relabelled as failed. The three observed-EOL cells provide 331 labelled records up to and including EOL (125 + 109 + 97). Records after the first crossing (43, 59 and 35 for B0005, B0006 and B0018, respectively; 137 in total) are excluded even if capacity temporarily recovers. Fig. 1 shows the raw capacity trajectories of all four cells.

C. Missing Values

Across the 636 original rows, the rounded missing fraction is 0.09 for R_e, R_{ct} and the EIS age, and 0.00 for all other columns (cycle, capacity, voltage, current, temperature, discharge duration, charge duration and charge voltage). These are overall rounded rates and not exact per-fold counts.

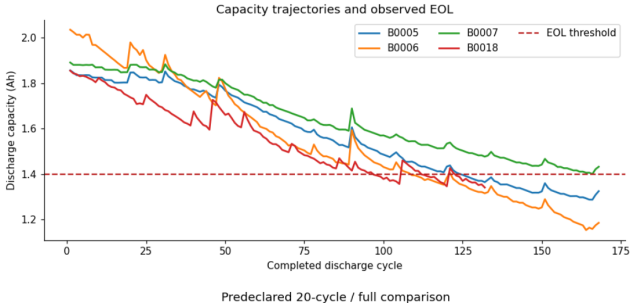


Fig. 1. Measured discharge-capacity trajectories for the four NASA cells (B0005, B0006, B0007, B0018). The dashed line denotes the 1.4 Ah end-of-life threshold. Three cells exhibit an observed crossing; B0007 remains right-censored under this definition. All recorded cycles are shown for context.

D. Target Definition

For cell b, let E_b be the first observed discharge cycle with measured capacity at or below 1.4 Ah. The target is $\mathrm{RUL}(b,t) = E_b - t$ for $t \leq E_b$. EOL is defined from raw measurements and not from a smoothed or extrapolated trajectory. The target counts observed discharge cycles, not calendar time or equivalent full cycles. A threshold crossing that might lie between sparse measurements is not inferred.

E. Evaluation Endpoints

Common endpoints begin at cycle 30, the largest window length, so that changing the history length does not change which cases are scored. B0005 contributes 96 scored endpoints, B0006 80 and B0018 68, a total of 244 per model and configuration. Earlier cycles supply context but are not scored.

F. Feature Groups

The full feature set contains 14 signals in code order: `cycle`, `capacity_ah`, `voltage_mean`, `voltage_std`, `voltage_min`, `current_mean`, `temperature_mean`, `temperature_max`, `duration_s`, `charge_duration_s`, `charge_voltage_mean`, `re_ohm`, `rct_ohm` and `eis_age_cycles`. The reduced set removes `capacity_ah`, `duration_s` and `charge_duration_s`, which can act as proxies for capacity, and retains the cycle index and the remaining signals. The cell identifier, the cell's lifetime, its final capacity and any normalized lifetime fraction are never used as predictors.

V. METHODOLOGY

The experimental framework compares a simple reference against increasingly flexible learners under a protocol in which every scored battery is withheld from training and tuning.

A. Time of Prediction

Prediction occurs after cycle t completes. Current capacity, voltage and current summaries and the discharge duration of that cycle are therefore available as inputs. This is a legitimate

definition for the task as specified but is not a forecast made before the current measurements exist. A beginning-of-cycle forecast would require lagging these inputs and is not claimed. The ground-truth EOL is used only offline to create the target.

B. Evaluated Models

1) **Cycle reference:** predicts $\max(0, \bar{E}_{\text{train}} - t)$, where $\bar{E}_{\text{train}}$ is the mean EOL of the training cells. Each training cell contributes once to this mean, so long-lived cells do not dominate it. It uses neither the feature set nor the sensor history, and therefore isolates the benefit of the learned models.
2) **Random Forest:** a bootstrap ensemble of regression trees [4], with depth and minimum leaf size chosen from two candidates on inner validation cells.
3) **XGBoost:** gradient-boosted trees with squared-error loss [5], conservative depth and learning rate, and deterministic CPU histogram trees.
4) **LSTM:** one causal, unidirectional LSTM layer [6] followed by a small dense regression head, trained on the complete standardized sequence.

A regularized linear (Ridge) control and a capacity-only feature set are implemented in the code but were not evaluated in the reported NASA run.

C. Windows and Representations

A window contains cycles $t - w + 1$ through t for $w \in \{10, 20, 30\}$, so only current and past cycles are used. The tree models receive four summaries of each retained standardized signal: the last value, the mean, the standard deviation and the ordinary least-squares slope across the window. With all 14 signals retained, the full tree input has 56 columns, and the actual count varies with fold-specific filtering. Feature names carry a suffix such as `__last`, `__mean`, `__std` or `__slope`. The LSTM receives the retained sequence instead. This compares sensible model-specific representations of identical historical observations and not identical hypothesis classes.

D. Preprocessing

Within every training split, signals with no finite values or with variance at most 10^{-12} are dropped, the remaining values are median-imputed and then standardized. These transformations are fitted on the flattened training-window timesteps only and applied to the held-out cell. Because windows overlap, interior timesteps receive repeated weight in this fit. This is an intentional window-based fit and does not use held-out observations. No bidirectional smoothing, interpolation from the future, PCA or learned feature selection is used.

E. Nested Leave-One-Battery-Out Evaluation

The protocol is applied separately to each combination of window and feature set:

1) Hold out one observed-EOL cell completely as the outer test cell.
2) On the remaining cells, rotate one entire cell as inner validation.
3) Refit preprocessing for every inner training split, and score each candidate by its mean inner-cell RMSE, with validation cells weighted equally.
4) Refit the chosen candidate and its preprocessing on all outer-training cells.
5) Score the untouched outer cell, and repeat for each observed-EOL cell.

The LSTM uses the epoch count and hidden size selected on inner cells, and the outer test cell never determines when training stops. In addition, a separate tree-selection procedure compares Random Forest and XGBoost across window and feature-set combinations using only inner validation scores; its outer error is saved separately. This provides an outer estimate of a validation-driven selection procedure, which is distinct from the post-hoc lowest entry in the fixed-configuration table.

F. Evaluation Metrics

The mean absolute error (MAE), root mean squared error (RMSE) and coefficient of determination (R^2) are used:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}, \tag{1}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{2}$$

Predictions are clipped only at zero. R^2 can be negative when a held-out cell is poorly predicted. Bias is the mean signed residual, and a positive bias indicates overestimated remaining life. Metrics are computed per held-out cell, averaged over seeds within each cell (one seed here), and then macro-averaged across cells. The between-cell RMSE standard deviation is a descriptive measure of heterogeneity and not a confidence interval. Overlapping windows are not independent samples, and no significance test is applied.

G. Feature Ablation

The grouped ablation compares the *full* and *reduced* sets at every window length and for every model. Because the reduced set removes capacity and both duration signals simultaneously, any difference is attributed to the group and not to capacity alone, and other retained signals may still correlate with capacity.

H. SHAP

Tree SHAP [7], [8] is applied to the inner-selected tree model for each held-out cell. The explainer uses an interventional feature perturbation with a deterministic background of up to 80 outer-training windows and up to 80 evenly spread held-out windows, and additivity check enabled. Contributions are expressed in terms of RUL cycles for the raw tree prediction before non-negative clipping, and colors represent standardized feature values. SHAP attribution highlights features the fitted model relies on. The approach does not infer

TABLE II
CANDIDATE CONFIGURATIONS SEARCHED IN THE FULL-MODE GRID

Model	Candidate 1	Candidate 2
Random Forest	250 trees, depth 5, min leaf 3	250 trees, depth None, min leaf 5
XGBoost	250 est., depth 2, learning rate 0.04	250 est., depth 3, learning rate 0.04
LSTM	hidden 16, 60 epochs, learning rate 0.003	hidden 32, 100 epochs, learning rate 0.003

physical causation, and the correlated capacity, duration and voltage features can split attribution.

VI. IMPLEMENTATION DETAILS

A. Software

The pipeline is implemented in Python as a modular package with a command line interface (data extraction, audit, demonstration, experiments and inference) and a standalone notebook which contains the same functions as readable cells. Scikit-learn [13] provides preprocessing and the Random Forest, XGBoost [5] provides gradient boosting, PyTorch [14] provides the LSTM, and the SHAP library provides Tree SHAP. Module responsibilities are: `data.py` (extraction and targets), `features.py` (windows and transforms), `models.py` (model and grid declarations), `experiment.py` (tuning, evaluation, explanation and inference), `reporting.py` (tables, figures and HTML) and `download.py` (archive retrieval). The reported results were produced by running the notebook in Google Colab. The printed environment lists Python 3.13.16, NumPy 2.3.5, pandas 2.2.3, SciPy 1.17.0 and scikit-learn 1.8.0.

B. Run Configuration

The run used `MODE="nasa"`, `FULL_MODELS=True`, windows (10, 20, 30), feature sets (full, reduced), a single seed (42) and a capacity threshold of 1.4 Ah. Full candidate grids were used, and Random Forest, XGBoost and the LSTM were evaluated alongside the cycle reference. The LSTM runs on CPU with one torch thread and seeded, deterministic-algorithm settings; no GPU acceleration is implemented.

C. Model Candidates

Each model has two candidate configurations in the full grid (not a Cartesian product), listed in Table II. Both XGBoost candidates use `reg:squarederror`, CPU histogram trees, `subsample`=1, `colsample_bytree`=1, `n_jobs`=1 and the experiment seed.

D. LSTM Architecture and Training

The LSTM comprises one unidirectional LSTM layer whose output at the last time step is passed through Linear(hidden size, 16), a ReLU non-linearity, and Linear(16, 1). The input has shape (windows, history length, retained features). We train the model using the Adam optimizer [12] with weight decay 10^{-4}, a batch size of 32, mean-squared error on a normalized target, and a gradient norm clipping threshold of 1.0. We standardize the target using the training mean and $\max(\text{training standard deviation}, 1)$. We do not use dropout, attention, bidirectionality, early stopping, or pre-trained weights.

E. Experiment Accounting

The completed protocol comprises 3 windows $\times$ 2 feature sets $\times$ 3 held-out cells = 18 experiment blocks, each of which scores four methods including the reference, giving 72 per-cell metric records summarized into 24 model/feature/window rows. Each of the three learned models has two candidates and each candidate uses two inner folds, giving 216 inner fits and 54 final outer-training fits, a total of 270 learned-model fits implied by the protocol. These are code-derived counts, not profiler measurements. Each inner fit trains on one battery and validates on another, and the number of windows does not increase the number of independent batteries.

VII. RESULTS AND DISCUSSION

The results below are taken from the stored outputs of the executed notebook. All metrics are equal-cell macro averages over B0005, B0006 and B0018 with one seed. MAE, RMSE and the between-cell standard deviation are in discharge cycles, and R^2 is unitless.

A. Overall Model Comparison

Table III lists all 24 recorded summary rows. The lowest macro RMSE is obtained by Random Forest with full features and a 20-cycle history (RMSE 8.986, MAE 7.637, R^2 0.838). XGBoost with the same settings is closely matched (RMSE 9.002, MAE 7.675, R^2 0.843). The R^2 of XGBoost is slightly higher despite its slightly higher RMSE because the metrics are averaged separately per cell. The RMSE difference of 0.016 cycles is too small to support a statement of superiority, and the two tree models are described as closely matched.

Relative to the cycle reference (RMSE 14.063, MAE 13.788), Random Forest with full features and a 20-cycle history reduces macro RMSE by approximately 36.10% and macro MAE by approximately 44.61%. These are relative reductions in error and not percentage accuracy. As the lowest entry in the table, this configuration is a descriptive result of the evaluated configurations and not an unbiased estimate of choosing a winner after seeing all test scores.

The LSTM provides a meaningful sequence-model comparison. Its lowest error appears at the full/10 configuration (MAE 9.121, RMSE 11.440, R^2 0.681), approximately 18.65% below the baseline RMSE. The compact LSTM therefore learns useful temporal information, while tree models on window summaries are more effective in this limited-cell experiment. This should not be read as an inherent weakness of LSTMs, since more diverse trajectories, tuning or alternative representations could change the comparison. Fig. 2 shows the predeclared 20-cycle, full-feature comparison.

TABLE III
COMPLETE SUMMARY OF THE NASA RUN (EQUAL-CELL MACRO AVERAGES, SEED 42). MAE, RMSE AND SD IN CYCLES; SD IS THE BETWEEN-CELL RMSE STANDARD DEVIATION

Model	Feat.	W	MAE	RMSE	SD	R^2
cycle_baseline	full	10	13.788	14.063	10.447	0.474
cycle_baseline	full	20	13.788	14.063	10.447	0.474
cycle_baseline	full	30	13.788	14.063	10.447	0.474
cycle_baseline	reduced	10	13.788	14.063	10.447	0.474
cycle_baseline	reduced	20	13.788	14.063	10.447	0.474
cycle_baseline	reduced	30	13.788	14.063	10.447	0.474
lstm	full	10	9.121	11.440	4.796	0.681
lstm	full	20	9.545	11.584	4.815	0.662
lstm	full	30	10.409	12.217	4.782	0.631
lstm	reduced	10	16.591	20.546	5.910	0.059
lstm	reduced	20	15.755	19.909	5.846	0.110
lstm	reduced	30	16.384	19.947	5.046	0.133
rf	full	10	9.654	11.638	6.206	0.646
rf	full	20	**7.637**	**8.986**	2.444	0.838
rf	full	30	7.979	9.567	0.606	0.825
rf	reduced	10	15.060	17.487	3.455	0.353
rf	reduced	20	11.621	14.210	2.307	0.579
rf	reduced	30	10.192	12.069	1.485	0.711
xgb	full	10	9.977	11.236	4.262	0.715
xgb	full	20	7.675	9.002	1.733	0.843
xgb	full	30	8.847	10.254	1.000	0.804
xgb	reduced	10	14.120	16.573	3.092	0.422
xgb	reduced	20	11.127	13.975	2.597	0.584
xgb	reduced	30	9.236	11.496	0.687	0.743

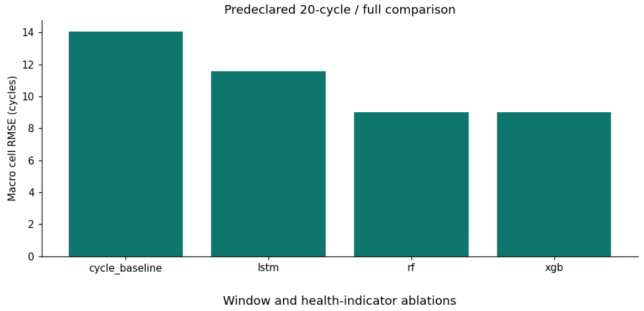


Fig. 2. Macro-averaged held-out-cell RMSE for the cycle baseline, LSTM, Random Forest and XGBoost using full health indicators and a 20-cycle history (seed 42). Random Forest and XGBoost achieve similar errors of 8.986 and 9.002 cycles, respectively. No uncertainty intervals are shown.

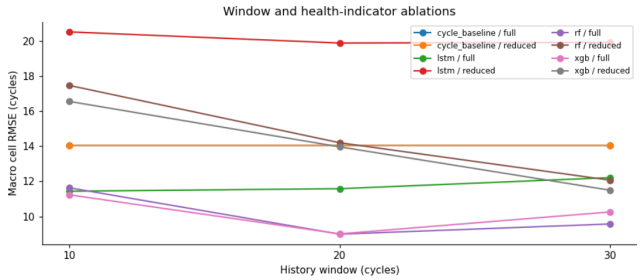


Fig. 3. Effect of history length and feature availability on macro held-out-cell RMSE (seed 42, common endpoints from cycle 30). Full-feature tree models favor a 20-cycle history, while longer histories improve the reduced-feature tree models. Curves are shown for the cycle baseline, LSTM, Random Forest and XGBoost with full and reduced features.

B. Temporal Context and Feature Ablation

Fig. 3 plots the macro RMSE against the history length for all eight model/feature combinations using common endpoints from cycle 30. A 20-cycle history proves optimal for the full-feature tree models: Random Forest RMSE drops from 11.638 to 8.986 to 9.567 for windows 10, 20 and 30, and XGBoost from 11.236 to 9.002 to 10.254. More history is therefore not automatically better. For the reduced-feature tree models, long histories prove beneficial consistently: Random Forest improves from 17.487 to 14.210 to 12.069, and XGBoost from 16.573 to 13.975 to 11.496. XGBoost with reduced features and a 30-cycle history has RMSE 11.496 and R^2 0.743, about 18.25% below the baseline, which shows that indirect signals with longer histories also retain predictive value when the direct capacity and duration inputs are removed. The full LSTM reaches its minimum at window 10 (11.440, 11.584 and 12.217 for windows 10, 20 and 30). The reduced LSTM (RMSE 20.546, 19.909 and 19.947) does not improve on the cycle baseline, which indicates the feature availability as an important design condition for the implemented sequence model. The two baseline curves overlap since the baseline does not depend on the feature set or window.

Full features dominate reduced features at every matched model and window. At a window of 20 cycles, the RMSE improvement from full over reduced features is approximately 36.76% for Random Forest, 35.58% for XGBoost and 41.82% for the LSTM. As the reduction removes capacity, discharge duration and charge duration together, this quantifies the dependence on the specified input bundle and is not a causal contribution of one sensor.

C. Inner-Selected Tree Procedure

Table IV reports the separate procedure in which the tree family, window and feature set are chosen using inner validation error only. It selected full features and a 10-cycle history in every fold: XGBoost for B0005 and Random Forest for B0006 and B0018. Their outer RMSE values are 12.104, 5.522 and 17.931 cycles, giving a macro RMSE of approximately 11.852 cycles, a mean MAE of 10.166 and a mean R^2 of approximately 0.639 (derived from the rounded rows). This is approximately a 15.72% macro-RMSE reduction over the baseline. It is a separate result from the post-hoc lowest configuration (8.986 cycles): the latter is the best fixed-configuration benchmark, whereas the former is an outer estimate of a validation-driven selection procedure. Both are reported. The hyperparameters chosen inside each fold are not printed in the notebook and are therefore not stated.

TABLE IV
INNER-SELECTED TREE PROCEDURE (SEED 42, FULL FEATURES, 10-CYCLE HISTORY IN EVERY FOLD)

Test cell	Model	Inner RMSE	MAE	RMSE	R^2	Bias	n
B0005	xgb	11.656	10.995	12.104	0.809	−10.952	96
B0006	rf	14.367	4.182	5.522	0.943	2.835	80
B0018	rf	8.864	15.320	17.931	0.165	14.816	68

TABLE V
PER-CELL DIAGNOSTICS AVERAGED OVER THE SIX WINDOW/FEATURE CONFIGURATIONS OF EACH MODEL (CYCLES)

Model	Test cell	MAE	RMSE	Bias
cycle_baseline	B0005	19.365	20.188	−19.365
	B0006	2.000	2.000	2.000
	B0018	20.000	20.000	20.000
lstm	B0005	8.140	11.151	−4.939
	B0006	13.821	16.336	−13.008
	B0018	16.942	20.334	16.351
rf	B0005	10.355	12.361	−8.334
	B0006	8.108	9.732	−1.641
	B0018	12.609	14.886	11.118
xgb	B0005	10.141	12.042	−9.558
	B0006	8.456	10.472	−1.622
	B0018	11.894	13.754	9.762

D. Cross-Cell Behavior

Table V gives per-cell diagnostics averaged over the six window/feature configurations of each model (and the single seed). They are not per-cell metrics of Random Forest with full features and a 20-cycle history nor of any single configuration. A positive bias indicates an overestimated RUL. The cycle baseline performs exceptionally well on B0006 (RMSE 2.000) because the lifetime of that cell just happens to be close to the mean of the other training cells; this is an informative reference and no sign that a learned model has to win on every cell. B0018 tends to receive optimistic remaining-life forecasts (positive bias for all learned models), and B0005 tends toward underestimation.

Fig. 4 shows predicted RUL and signed residuals for the full-feature, 20-cycle models on each held-out cell. B0005 predictions tend to start below the true RUL, with changing local residual patterns for Random Forest and XGBoost. On B0006 the cycle baseline is unusually close, while the LSTM underestimates much of mid-life. On B0018 the learned models tend toward positive residuals later in life, with larger late-life overestimation by the LSTM. These curves are read qualitatively, since the notebook does not print an exact residual table for them. The cell-dependent patterns point to where calibration can be improved and complement the aggregate comparison.

E. SHAP Feature Attribution

Fig. 5 shows Tree SHAP beeswarm plots for the inner-selected models (full features, 10-cycle history, seed 42), using the top 15 summary features. These are not explanations of the Random Forest with full features and a 20-cycle history. The rankings below are transcribed visually from the plots and are not numerical importance values.

- **B0005 (XGBoost):** mean discharge duration (`duration_s__mean`) has the largest attribution spread, followed by mean capacity, the current cycle index, the latest discharge duration and mean voltage. Larger durations are generally associated with positive RUL contributions and smaller durations with negative ones.
- **B0006 (Random Forest):** mean capacity (`capacity_ah__mean`) leads, followed by mean voltage and the latest capacity. Higher mean capacity generally contributes toward larger predicted RUL.
- **B0018 (Random Forest):** mean current, mean maximum temperature and mean charge voltage lead, followed by mean capacity. Signed current values are preserved, so a higher numeric current should not be read as a larger current magnitude.

The leading features therefore differ across held-out cells, which indicates fold-dependent fitted relationships and motivates validation across more cells and operating conditions. Because correlated capacity, duration and voltage features can share attribution and the selected model differs by fold, this is not a universal sensor ranking and is not evidence of physical causation.

F. Inference Demonstration

A saved Random Forest (full features, 10-cycle history, seed 42, trained with B0005 excluded) was loaded and given only the first 40 cycles of B0005. It returned a predicted RUL of 68.627 cycles at cycle 40 (status "model forecast", `seen_training_cell`=False). With the audited EOL at cycle 125, the retrospective true RUL at cycle 40 is 85 cycles, so the prediction underestimates by 16.373 cycles. This single example demonstrates model persistence, reuse of the fitted preprocessing and inference without supplying the future endpoint to the predictor. It does not replace the held-out metric tables.

G. Comparative Discussion

Taken together, the results suggest that a leakage-aware pipeline with fully used health indicators and tree models on the windowed summary statistics substantially outperforms the cycle reference on the held-out cells, with Random Forest and XGBoost being close to each other in performance. The worth of the context size depends on the information available to the model: indeed, a 20-cycle window is optimal if the direct indicators of the capacity and duration are present, while the larger window is preferable in the absence of direct features. The grouped ablation analysis suggests that all model families benefited from the use of the grouped features, specifically, the ones related to the capacity and the duration. Finally, the inner selected procedure with the tree ensemble had a slightly lower macro RMSE, equal to approximately 11.852 cycles, which serves as a conservative estimate of the performance achievable on the given task. The presence of cell-specific

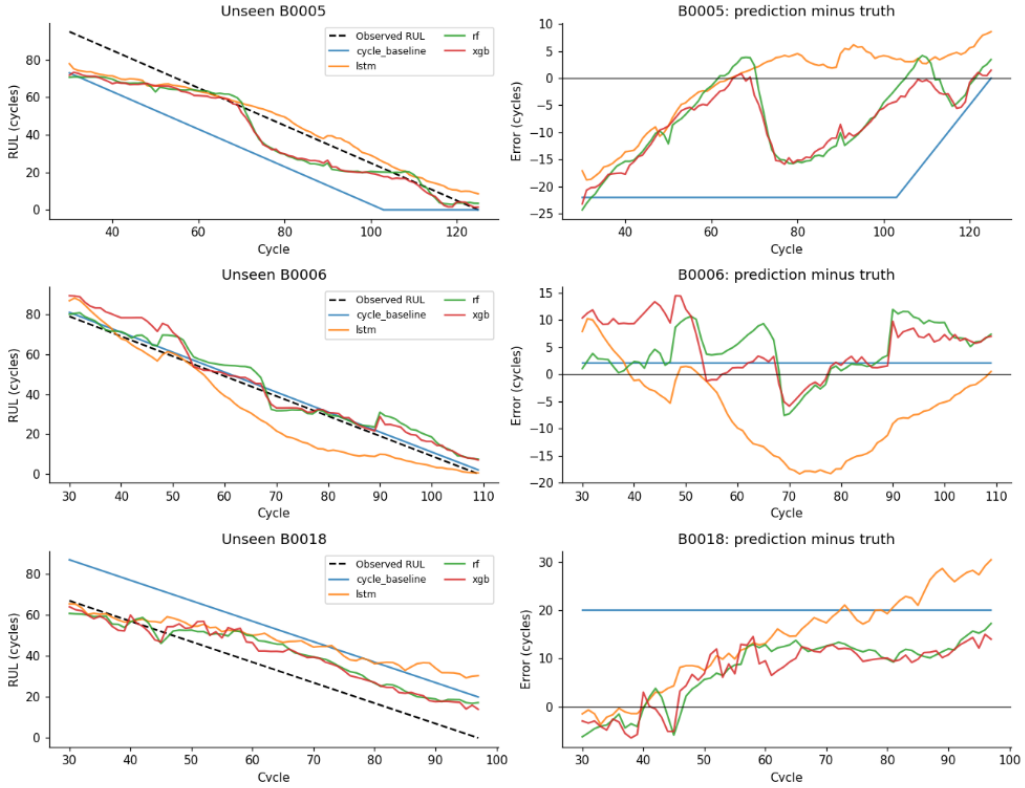
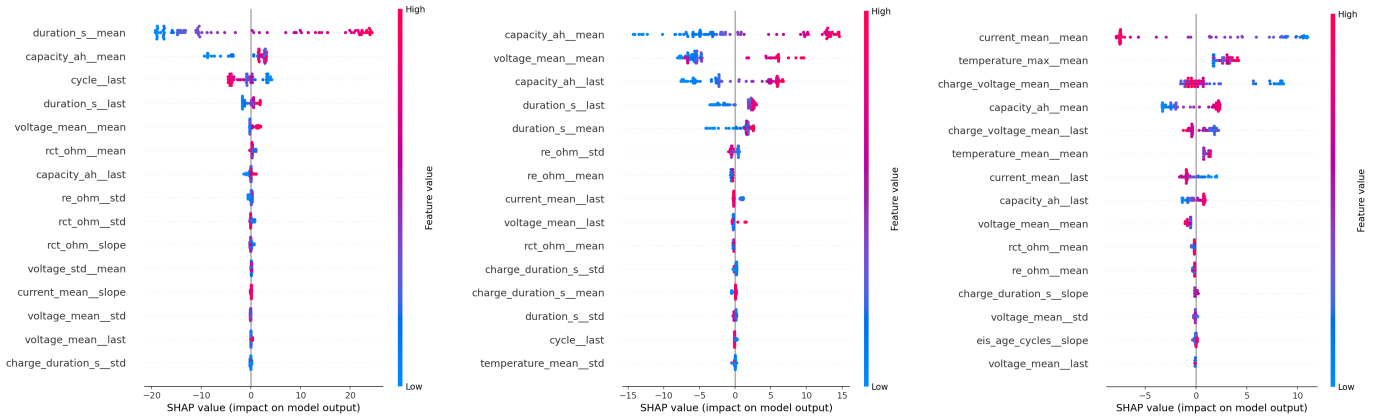


Fig. 4. Predicted and observed RUL (left) and signed residuals (right) for each held-out NASA cell (rows: B0005, B0006, B0018), using full features, 20-cycle histories and seed 42. The dashed line is the observed RUL; curves show the cycle baseline, LSTM, Random Forest and XGBoost. Positive residuals indicate optimistic remaining-life estimates. Each cell is excluded from the training and tuning data of its model (96, 80 and 68 endpoints, respectively).



(a) B0005, XGBoost (b) B0006, Random Forest (c) B0018, Random Forest

Fig. 5. Tree SHAP summary plots for the inner-selected models (full features, 10-cycle history, seed 42) on each held-out cell: (a) XGBoost on B0005 (80 of 96 endpoints explained), (b) Random Forest on B0006 (80 windows), (c) Random Forest on B0018 (68 windows). Horizontal position is the contribution to raw predicted RUL in cycles; colors indicate standardized feature values (red high, blue low).

bias and variability in the SHAP ranks between folds suggests that further improvements can be achieved by considering a more diverse set of cells and operating conditions.

H. Limitations

This is a carefully controlled small-battery benchmark, and its conclusions are strongest for the three observed-EOL trajectories and the after-discharge prediction definition. Several scope limits define how the results should be read and point to natural extensions.

- **Number of scored cells.** Only three cells are scored, and in the inner loop each fit can contain a single training cell. Between-cell standard deviations are descriptive, and the results do not support statistical superiority claims, population confidence intervals or significance tests.
- **Single seed.** Only one seed (42) was run. Additional

seeds would measure algorithmic variation and not diversity of unseen batteries.

- **Censoring.** B0007 does not cross 1.4 Ah and is excluded from supervised scoring, so conclusions apply to observed-EOL cells and may carry selection bias.
- **Prediction time.** Current capacity is available because prediction occurs after the discharge, so the results do not describe a forecast before that measurement exists.
- **Post-hoc best entry.** The lowest table entry is descriptive, and the inner-selected tree procedure should be used as the estimate for a validation-driven selection.
- **Ablation interpretation.** The full/reduced comparison quantifies dependence on a bundle of inputs, and SHAP describes associations learned by the fitted models.
- **Representation.** Trees and the LSTM use different representations of the same history, and the LSTM is compact and untuned beyond two candidates.
- **Uncertainty.** No uncertainty intervals, external datasets, survival models or deployment testing are included, and no real-world reliability guarantee follows from the results.

VIII. Conclusion and Future Scope

This study has constructed and evaluated a controlled comparison of RUL regression approaches on the NASA battery data set with explicit censoring audit, past-only windows, common evaluation endpoints, training-fold-only preprocessing and nested battery-level validation. Random Forest with full features and a 20-cycle history achieved the lowest macro RMSE (8.986 cycles) and MAE (7.637 cycles), closely followed by XGBoost (9.002 cycles), against 14.063 cycles for the cycle reference. The validation-driven tree-selection approach achieved around 11.852 cycles, a more conservative outer estimate. The use of full health indicators outperformed reduced indicators at every matched model and window, longer histories improved reduced-feature tree models, and Tree SHAP demonstrated fold-dependent attribution patterns that are best interpreted as associations. All error values are expressed in discharge cycles, and the findings are descriptive for three scored cells and one seed.

This study will pursue the evaluation of additional cells and operating protocols, repeated seeding, the incorporation of external validation and evaluation of the implemented Ridge and capacity-only controls; learning from censored cells with survival-aware methods; calibrated uncertainty estimates; and beginning-of-cycle forecasts with lagged inputs as extensions and are not claimed as complete features.

Acknowledgment

Assistance from an AI system (Claude, developed by Anthropic) was used in building the code package and notebook and in developing a report structure, as required to be acknowledged by the course. The author executed the notebook on the NASA data in Google Colab.

Appendix

The experimental evidence was taken from the stored outputs of the executed notebook (SHA-256 of the reviewed notebook: `b6617d59...693dc5`; run identifier `notebook_nasa_20261012_012124_108410`, which is an identifier and not a measured runtime). The saved output directory is not bundled with this report, so exact chosen hyperparameters, resolved XGBoost, PyTorch and SHAP versions, source hashes, LSTM loss curves, stage-specific errors, total runtime and hardware details are not reported here. They can be obtained from the saved result files (`tuning.json`, `environment.json`, loss records and error tables) if required.

During setup, the surrounding Colab environment noted a dependency conflict between a package not used by this project, and SHAP output three non-fatal warnings about future random-number behavior in plotting. Training, SHAP plots and inference completed, and these messages are recorded only as environment notes. A dedicated environment is preferable for repeat runs, and the original software tests were run in the build environment, with no test log attached from the Colab execution.

References

[1] B. Saha and K. Goebel, "Battery Data Set," NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, 2007. [Online]. Available: https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/

[2] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Inf. Sci.*, vol. 191, pp. 192–213, 2012.

[3] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *J. Mach. Learn. Res.*, vol. 11, pp. 2079–2107, 2010.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.

[8] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020.

[9] K. A. Severson *et al.*, "Data-driven prediction of battery cycle life before capacity degradation," *Nature Energy*, vol. 4, pp. 383–391, 2019.

[10] R. R. Richardson, M. A. Osborne, and D. A. Howey, "Gaussian process regression for forecasting battery state-of-health," *J. Power Sources*, vol. 357, pp. 209–219, 2017.

[11] Y. Zhang, R. Xiong, H. He, and M. G. Pecht, "Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries," *IEEE Trans. Veh. Technol.*, vol. 67, no. 7, pp. 5695–5705, 2018.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[13] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[14] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019.